

GUO, XI

xiguo@uw.edu — +1 (406)402-2751 — www.linkedin.com/in/xiguo815

EDUCATION

University of Washington

Master of Science in Electrical and Computer Engineering

Seattle, WA

Sep 2026 — Jun 2028

Huazhong University of Science and Technology

Bachelor of Engineering in Computer Science and Technology

Wuhan, China

Sep 2019 — Jun 2023

SKILLS

Languages & Frameworks: C/C++, Python, Go, Java, Bash, PyTorch, TensorFlow, TorchRec, CUDA, Triton

Systems & Tools: Distributed Training, Parameter Server, Zookeeper, HDFS, Linux (Kernel/Profiling), Docker, Kubernetes

PROFESSIONAL EXPERIENCE

PDD Holdings (Temu)

Software Engineer, Machine Learning Infrastructure

Shanghai, China

Jul 2023 — Nov 2025

Scalable Training for Generative Recommendation Models

- Implemented training workflows for genrec models (HSTU) using PyTorch and TorchRec, integrating DDP and DMP to scale billion-parameter sparse training on 300M+ DAU datasets for multi-task workflows (recall, CTR, CVR).
- Built a pipelined training execution framework to overlap CPU-side data preprocessing, sparse embedding lookup, forward/backward computation and parameter updates, improving training throughput by 40%+.
- Optimized sparse embedding communication by deduplicating feature IDs before TorchRec embedding table lookup, reducing all-to-all traffic and improving 32-GPU end-to-end training throughput by 25%+.

Training Framework Migration

- Migrated a recommendation model training pipeline from TensorFlow to PyTorch on multi-node H100 clusters.
- Integrated custom C++ sparse Parameter Server as a CPU backend to offload massive read-only embedding tables, effectively bypassing GPU memory limits for memory-intensive downstream models.
- Re-implemented the end-to-end training lifecycle including sharded checkpointing, high-throughput data prefetching, and cross-framework consistency checks, to match existing training workflows.
- Optimized training efficiency through data prefetching and CPU-GPU coordination, leading to 30%+ higher GPU utilization and 15% faster training compared to the legacy TF system.

Semi-Synchronous Training Mechanism

- Extended SyncReplicaOptimizer (TensorFlow v1.0) with a semi-synchronous mechanism, enabling non-blocking worker execution without blocking to improve system throughput while preserving convergence stability.
- Implemented token-based gradient admission and staleness control on the parameter server to aggregate gradients at a fixed global batch size while filtering stale updates.
- Improved 60%+ training throughput over native synchronous training on recommendation CVR model, reaching 85% of asynchronous efficiency with +0.3% AUC improvement.
- Supported parameter-level update strategies via TensorFlow graph modifications to meet model-specific requirements.

Distributed Training Coordination System

- Designed a coordination mechanism for multi-node startup synchronization and runtime metadata distribution.
- Implemented leader election (Zookeeper + CAS) to assign a primary worker which handles code distribution and cluster metadata collection, eliminating external dependencies during job startup.
- Coordinated worker readiness and generated distributed configurations to enable reliable cluster bring-up.

Machine Learning Platform Backend

- Contributed to backend development of ML platform, supporting task submission, scheduling, and monitoring.
- Developed RESTful APIs (Python/Java) for training jobs, resource management, and user access control.
- Unified inconsistent API endpoints across services by implementing a proxy-based routing layer, and validated consistency by comparing proxy and direct API responses in production.

Sensetime

Software Engineer Intern

Shanghai, China

Jul 2022 — Sep 2022

- Contributed to backend development of a deep learning platform supporting training and deployment workflows.
- Developed and maintained backend services using Golang (Gin), supporting internal microservices and APIs.
- Worked on training job scheduling and deployment using Kubernetes, improving resource utilization and job stability.